TryHackMe – Brooklyn Nine Nine

Penetration Testing Walkthrough Report

Prepared by: Manoj Bora
Platform: TryHackMe | Room: Brooklyn Nine Nine | Difficulty: Easy
Date: 25 September 2026

---

1. Objective

This report covers my walkthrough of the TryHackMe room "Brooklyn Nine Nine", an easy Linux boot2root box themed around the Brooklyn 99 TV show. The room has two intended routes to root. I went with the steganography route — enumerate the target, find a foothold hidden inside an image, get user access, capture the user flag, then escalate to root through a sudo misconfiguration.

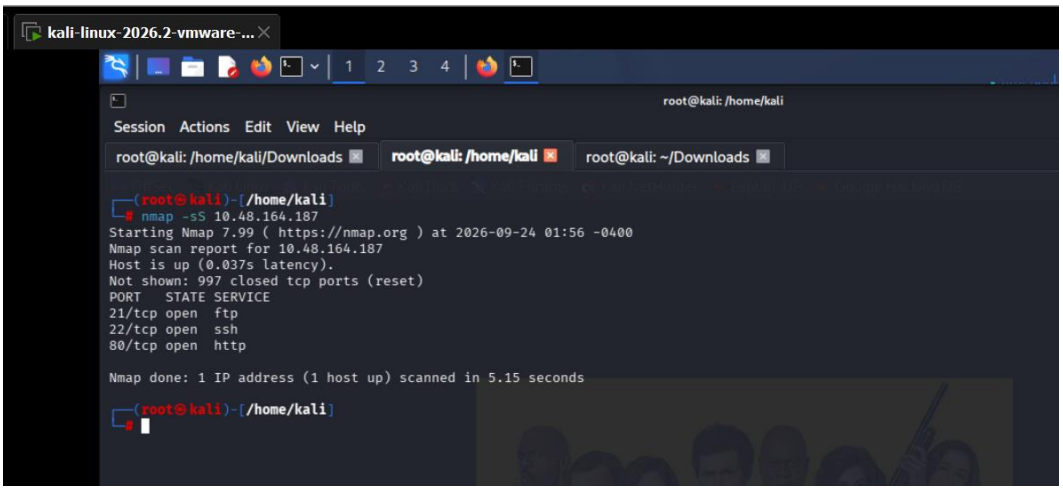
2. Tools Used

- Nmap — port scanning
- Firefox (view-source) — manual web inspection
- StegSeek — steganography passphrase cracking
- Steghide — steganography extraction
- SSH — remote access
- sudo -l — privilege enumeration
- GTFOBins — privilege escalation reference
- nano (abused via sudo) — privilege escalation vector

3. Walkthrough

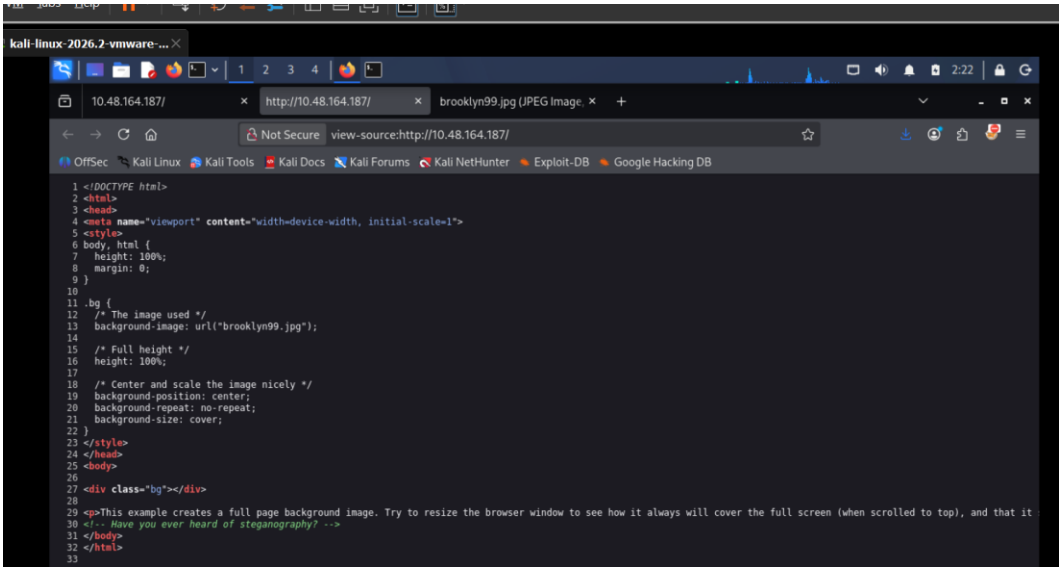
Reconnaissance

1. Ran an Nmap SYN scan against the target (nmap -sS <target-ip>). 3 ports came back open — 21 (FTP), 22 (SSH), 80 (HTTP).

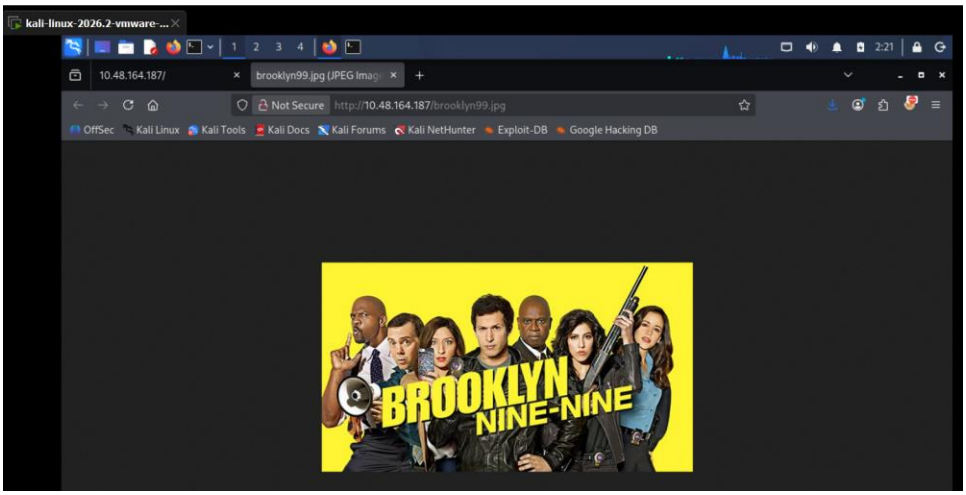


Web Enumeration

2. Port 80 was open, so checked the page in the browser first, then looked at the page source (view-source). Found an HTML comment: "Have you ever heard of steganography?" The CSS also referenced a background image, brooklyn99.jpg.



3. Browsed directly to the image at /brooklyn99.jpg. It was just the Brooklyn 99 cast photo — but the comment made it clear the image itself was worth digging into.

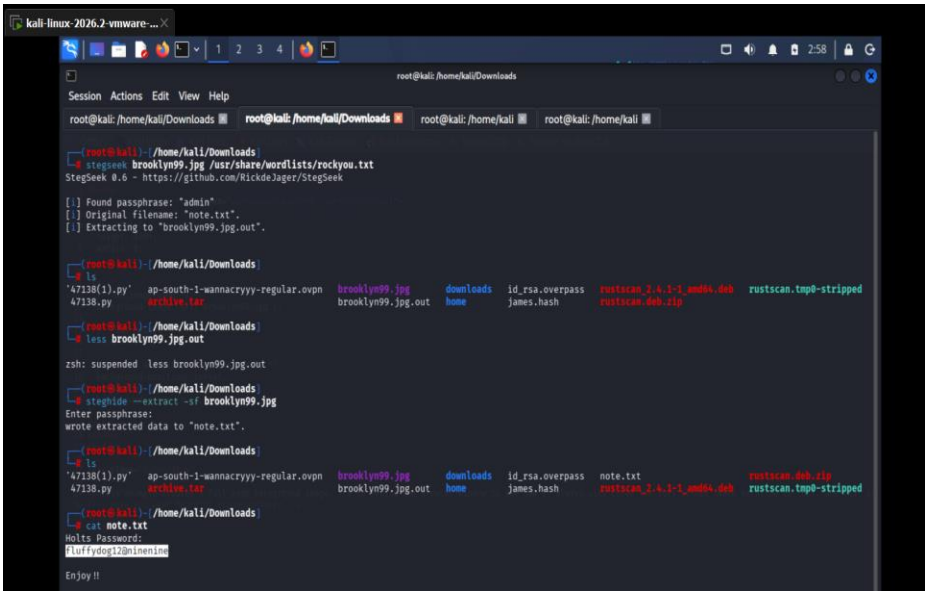


Steganography

4. Downloaded brooklyn99.jpg and ran StegSeek against it with the rockyou.txt wordlist (stegseek brooklyn99.jpg /usr/share/wordlists/rockyou.txt). It cracked the passphrase — "admin" — and reported the original hidden filename as note.txt, auto-extracting it to brooklyn99.jpg.out.

5. Re-ran steghide --extract -sf brooklyn99.jpg with the cracked passphrase to pull the hidden file out properly, giving note.txt.

6. Read note.txt — it contained a plaintext credential: Holt's Password: fluffydog12@ninenine.



Gaining Access (SSH)

7. SSH'd in as holt using the password from note.txt (ssh holt@<target-ip>) — login succeeded straight away.

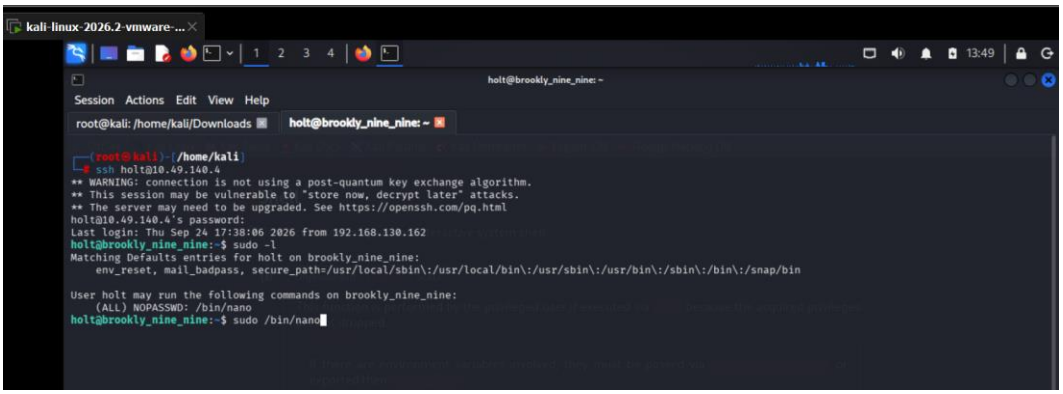
8. Listed the home directory and read user.txt to get the user flag.

```
┌──(root㉿kali)-[/home/kali]
└─# ssh holt@10.48.133.158
The authenticity of host '10.48.133.158 (10.48.133.158)' can't be established.
ED25519 key fingerprint is: SHA256:ceqkN71gGrXeq+J5/dquPWgcPWwTmP2mBdFS2ODPZZU
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:11: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.48.133.158' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
holt@10.48.133.158's password:
Last login: Tue May 26 08:59:00 2020 from 10.10.10.18
holt@brookly_nine_nine:~$ ls -la
total 48
drwxr-xr-x 6 holt holt 4096 May 26  2020 .
drwxr-xr-x 5 root root 4096 May 18  2020 ..
-rw------- 1 holt holt   18 May 26  2020 .bash_history
-rw-r--r-- 1 holt holt  220 May 17  2020 .bash_logout
-rw-r--r-- 1 holt holt 3771 May 17  2020 .bashrc
drwx------ 2 holt holt 4096 May 18  2020 .cache
drwx------ 3 holt holt 4096 May 18  2020 .gnupg
drwxrwxr-x 3 holt holt 4096 May 17  2020 .local
-rw-r--r-- 1 holt holt  807 May 17  2020 .profile
drwx------ 2 holt holt 4096 May 18  2020 .ssh
-rw------- 1 root root  110 May 18  2020 nano.save
-rw-rw-r-- 1 holt holt   33 May 17  2020 user.txt
holt@brookly_nine_nine:~$ less user.txt

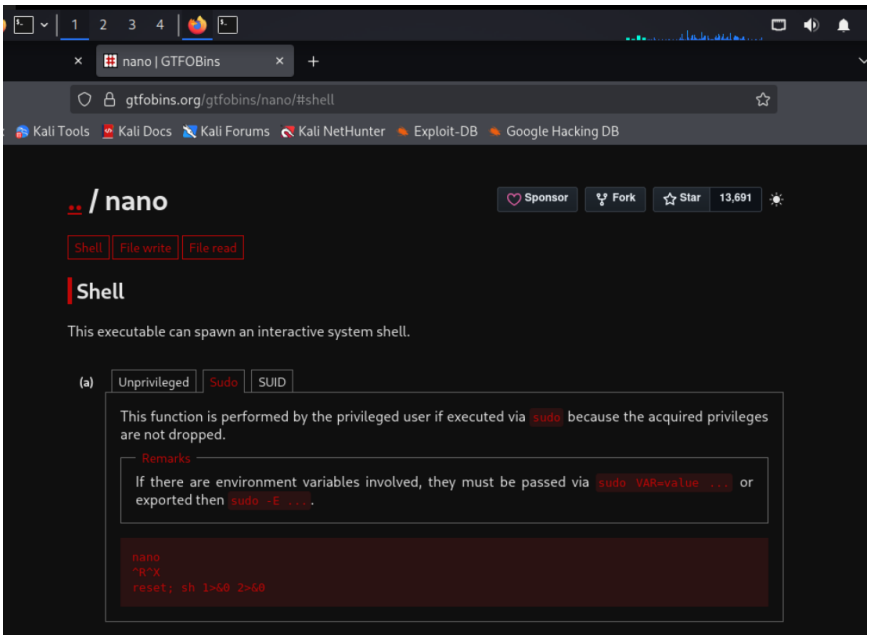
[1]+  Stopped                 less user.txt
holt@brookly_nine_nine:~$ cat user.txt
ee11cbb19052e40b07aac0ca060c23ee
holt@brookly_nine_nine:~$
```

Privilege Escalation

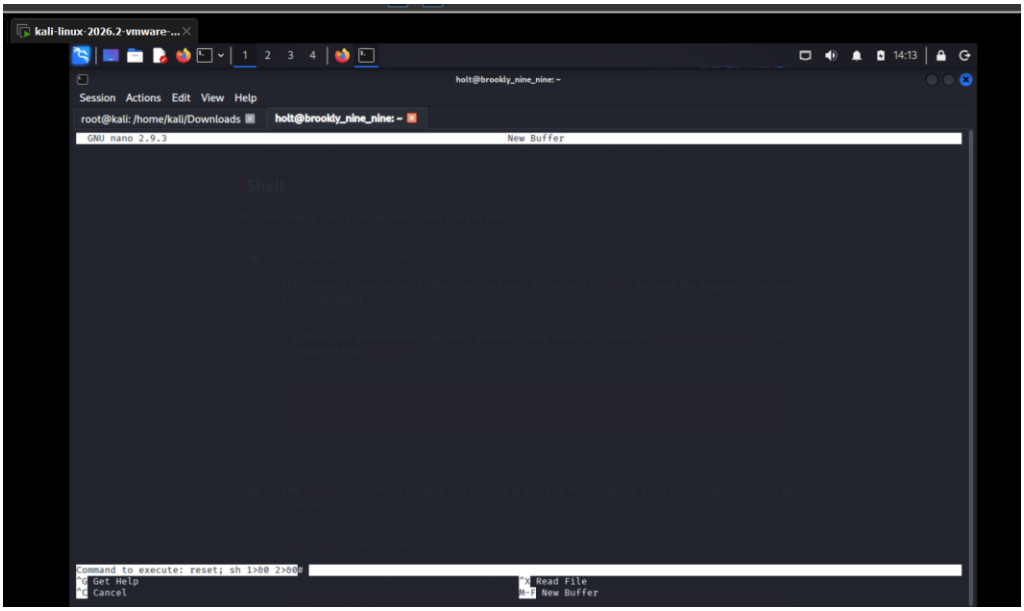
9. Checked sudo rights with sudo -l. holt was allowed to run /bin/nano as root with NOPASSWD — no restrictions on arguments.



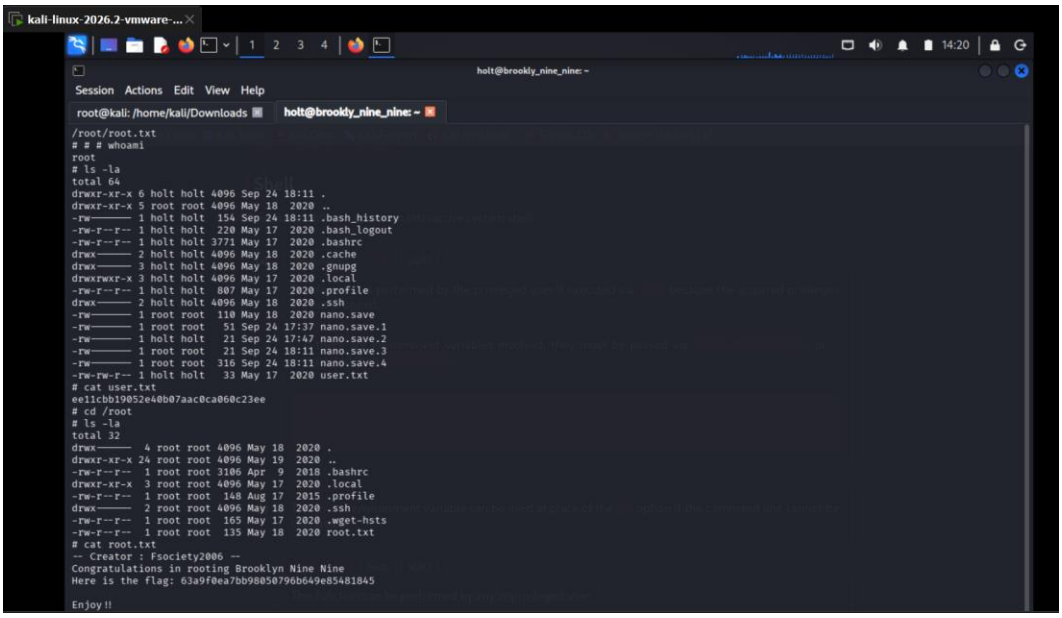
10. Looked up nano on GTFOBins. It confirmed nano can be abused for a shell escape: open nano, press Ctrl+R then Ctrl+X to bring up the "Execute Command" prompt, then run reset; sh 1>&0 2>&0 to spawn a shell with the caller's privileges.



11. Ran sudo /bin/nano, pressed Ctrl+R then Ctrl+X, and typed reset; sh 1>&0 2>&0 into the Execute Command prompt.



12. This dropped straight into a root shell. Confirmed with whoami — root. Navigated to /root and read root.txt for the root flag, signed by the room's creator, Fsociety2006.



4. Flags Captured

User Flag: ee11cbb19052e40b07aac0ca060c23ee

Root Flag: 63a9f0ea7bb98050796b649e85481845

5. Findings & Vulnerabilities

Summary of the weaknesses that made this box solvable, rated by how much impact they had on the compromise:

Finding	Severity
HTML comment on the web server hinted at steganography, with no access control stopping anyone from finding it	**Low**
Sensitive image was protected with a weak, wordlist-crackable steganography passphrase ("admin"), broken instantly with StegSeek	**Medium**
Hidden note stored a real account password in plaintext, and that same weak, guessable password was accepted directly over SSH	**High**
Sudo misconfiguration allowed a low-privileged user to run /bin/nano as root with no restrictions, giving an instant root shell via a known GTFOBins technique	**Critical**

6. Conclusion

Brooklyn Nine Nine was a short, beginner-friendly box built around steganography and a classic sudo/GTFOBins privilege escalation. The main takeaways were to always check page source and any hosted images for hidden clues, never store real credentials in a note protected only by a weak passphrase,

and to keep sudo rules tightly scoped — letting a user run an editor like nano as root with no restrictions is an instant path to full compromise.